

What testing the guardrail stack locally actually found

A local instance was run against the real seeded corpus — the Tamil Nadu Cooperation Department's 2024-25 Citizen Charter — and exercised across PII masking, prompt injection, content safety, and grounded retrieval. The rail logic itself scored perfectly. The headline finding is a latency-driven failure mode that silently drops real, correct answers under sustained load.

Every number below is measured on this running instance, not estimated
51 hand-built test cases, an automated eval suite (rewritten against the real corpus), and live traces with request IDs
Prepared 2026-08-29 for a lead review

What is in here

1. The one-line summary

What's solid, what isn't, and why it matters

2. What's solid — the rail logic

1.0 exact-verdict match across 39 rail cases, 12 PII entity types, 0 false positives/negatives

3. The headline finding — retrieval silently drops real answers

~70% of grounded questions refused under sustained load, with request IDs and timing

4. Root cause

A per-span judge fallback running serially, right at the edge of its own timeout budget

5. Secondary findings

A stale allowlist, a missing phone-number pattern, a test suite that predated the real corpus

6. What's already fixed

The eval suite is rewritten and reverified against the real corpus

7. Recommendation

What to change, in priority order

1 · The one-line summary

The guardrails work. The pipeline they sit inside does not reliably survive its own latency budget. Every deterministic and semantic check we could construct — PII detection across 12 entity types, prompt injection, leaked secrets, destructive intent, and all six content-safety categories — matched expectations exactly, with zero false positives and zero false negatives. But when the same corpus questions were run through one sustained process (the same shape as the real deployed server, not a fresh CLI call each time), a retrieval-side privacy scan started timing out often enough to make the assistant answer **"I don't have anything in the knowledge base that answers this"** to questions that have a real, correct, verifiable answer.

1.0 RAIL VERDICT MATCH (39/39 CASES)	0 / 0 FALSE POSITIVE / FALSE NEGATIVE RATE	9 / 13 GROUNDED QUESTIONS WRONGLY REFUSED	27–29s TYPICAL SCAN TIME VS. A 20–30S BUDGET
--	--	---	--

The risk in plain terms. A citizen asks a real, in-scope question. The correct answer exists in the document. The assistant tells them it doesn't know — not because the guardrails correctly judged the request unsafe, but because an internal privacy check ran out of time and the system discarded the context rather than risk leaking something unscanned. That is the safer failure mode by design, but it is failing far more often than the budget was sized for.

2 · What's solid — the rail logic

39 rail-only cases were run directly against the guardrail evaluation function — the same one every real request passes through — covering PII, prompt injection, leaked secrets, destructive intent, and all six content-safety categories. Every checksum used (SSN, credit card, Aadhaar via Verhoeff, PAN, IBAN) was independently verified against the project's own validators before being used in a test, so a "should mask" case is a case that genuinely passes its checksum, and a "should pass" case genuinely fails it.

CATEGORY	CASES	RESULT
PII — real identifiers (email, phone, SSN, card, Aadhaar, PAN, IBAN, IP, DOB, claim ref, name+address)	12	12/12 masked correctly
PII — false-positive controls (numbers that fail their checksum)	5	5/5 passed through
Prompt injection / jailbreak	4	4/4 blocked
Leaked secrets (API key, password, AWS key) + 1 false positive	4	4/4 correct
Destructive intent + self-service false positive	3	3/3 correct
Content safety — 6 hazard categories + false positives	8	8/8 correct
Meta / appeal questions (not mistaken for attacks)	2	2/2 correct

The automated evaluation suite (`eval/suite.yaml` , run via `python run.py --eval`) independently confirms this: **39/39 exact verdict match, false-positive rate 0.0, false-negative rate 0.0, all 10 configured detection kinds matched.** This part of the system is not a close call — it is correct across everything we could think to throw at it.

3 · The headline finding — retrieval silently drops real answers

13 questions that require the full pipeline — real corpus retrieval, generation, and grounding, not just a rail check — were run through one sustained process. **9 of 13 came back with zero retrieved context** and a generic refusal, even for questions independently confirmed to have a correct, specific answer in the document.

```
GR-01  chunks=1 → real answer  76.0s GR-02..06  chunks=0 → "not in the knowledge base" - 5 in a
row SCP-01..03  chunks=0 → same refusal - 3 more RET-01  chunks=1 → real answer  75.8s RET-02
chunks=1 → real answer  76.6s RET-03  chunks=0 → refusal  58.7s DES-04  chunks=0 → refusal
32.8s
```

Order as run, one continuous process — request IDs in the raw traces are available on request.

This is not a one-time cold-start problem — it recovers (RET-01 , RET-02) and then fails again (RET-03 , DES-04) later in the same run. It behaves like a coin that is landing on the wrong side roughly 7 times out of 10, in a system that is supposed to fail closed rarely, not routinely.

Concrete example. "What is the maximum amount available under a jewel loan?" has a real, exact answer in the document: **Rs. 30 Lakh at 11.50% interest.** In this run it came back as *"I don't have anything in the knowledge base that answers this"* — a factually wrong refusal, delivered with full confidence and a request ID, in 43.5 seconds.

4 · Root cause

Every question that retrieves context also runs a privacy scan (`pii.entities`) over the retrieved text before the model is allowed to see it — the real document is a branch directory dense with names, addresses, and phone numbers, so this is doing real work, not scanning empty space. That scan has two layers: a fast local recognizer (Presidio, "about a second of CPU" per the project's own documentation) and a slower fallback that asks the Claude judge to double-check anything Presidio is unsure about.

Measured directly, in isolation, that scan costs **27,000–29,000 milliseconds** — almost entirely the judge-fallback calls, run one after another, not in parallel. The system is configured to abandon an unfinished rail and discard whatever context it was protecting once a configurable time budget is exceeded — a *deliberate* and correct safety choice, a locked invariant that cannot be turned off. The problem is not that this exists; it's that the scan's real cost sits at 90–96% of whatever budget it's given, so ordinary network latency is enough to tip it over on a large fraction of calls.

LAYER	TYPICAL COST	NOTES
Presidio (local, no network call)	~1s	per the project's own configuration comments
Judge fallback (Claude API, per unmatched span, serial)	~1.7–4s <i>each</i>	dozens of spans in a dense chunk
Observed total, this corpus	27–29s	against a 20,000–30,000ms budget

Already tried and ruled out. Raising the timeout budget from 20,000ms to 30,000ms did not fix this — it just gave the same near-the-edge scan a slightly fairer coin flip. A genuine fix needs to either stop the judge fallback from running per-span in series on dense chunks, or accept the local-only recognizer's coverage for this specific surface.

5 · Secondary findings

5.1 · The department's own contact details get masked like a citizen's private data

The exemption list for "published, not private" contacts (`pii.allowlist`) only covers `*.municipal.gov.in` , `*.nic.in` , and toll-free numbers — written for an earlier, synthetic demo corpus. The real document's own institution uses `@tnscbank.com` , which isn't covered, so asking for the bank's own published email returns a masked placeholder instead of the real address.

5.2 · One real phone format isn't recognized as PII at all

Chennai-style landline numbers with no leading zero and no STD code (`2530 2300`) don't match any of the three configured phone patterns. They pass through completely unmasked — the opposite failure mode from 5.1, and worth knowing about for the same reason.

5.3 · The bundled evaluation suite predated the real corpus

The seeded document was swapped from a synthetic multi-topic demo set to the real Citizen Charter in a recent commit, but `eval/suite.yaml` was never updated to match. Its retrieval and answer-content sections scored **0.0** across the board — not because retrieval was broken, but because every expected document ID and fact referenced a topic (trade licences, housing grants, birth certificates) that no longer exists in the corpus. This is fixed — see §6.

6 · What's already fixed

`eval/suite.yaml` has been rewritten against the real corpus: every fact used was re-read from the source document and independently verified before being written into a test — loan eligibility, jewel-loan and mortgage-loan limits, branch counts. A new rail case was also added that masks a verbatim excerpt of the real document's own contact block, formalizing finding 5.1 as a regression test rather than leaving it undocumented.

SECTION	BEFORE	AFTER REWRITE
Retrieval (recall / precision / MRR / hit@1)	0.0 / 0.0 / 0.0 / 0.0	1.0 / 1.0 / 1.0 / 1.0
Rails (exact verdict match)	1.0 (38 cases)	1.0 (39 cases)

Two independent re-runs of the corrected suite's answer section still showed 2–3 of 5 questions occasionally failing — but investigating those live traces confirmed the cause is the same finding as §3 and §4, genuine variance in the retrieval-latency budget, not a new defect in the rewritten suite itself.

7 · Recommendation, in priority order

1. **Fix the retrieval-latency budget properly, not by raising a number.** Either cap/parallelize the judge fallback so a dense chunk's scan time stops scaling linearly with span count, or scope `pii.entity_engine` to Presidio-only on the retrieval surface specifically, accepting slightly weaker name/address coverage there in exchange for not silently dropping correct answers most of the time. This is the fix that actually moves the needle — everything else in this report is secondary to it.
2. **Update `pii.allowlist`** to cover the real corpus's own published domain, so the assistant can answer "who do I contact" the way it's supposed to.
3. **Add a phone-number pattern** for the Chennai-style landline format found in finding 5.2, so it stops being silently unmasked.
4. **Land the eval suite rewrite** (already done locally, described in §6) so CI reflects the real corpus going forward.

The reassuring part. None of this is a guardrail-logic problem. Every rail that was designed to catch something caught it, every time, across 39 adversarial and false-positive cases. The fix needed here is a performance and configuration fix to one specific rail on one specific surface — not a rethink of the approach.

GuardRails local testing briefing – prepared 2026-08-29. Reproduce with `python run.py --eval --answers` and `python run.py --ask "<question>"` against `config/policy.yaml` + `config/overrides.yaml`.